

Angular Moderno

25 - Reactive Forms



Aula 25 – Reactive Forms

Criação de Formulários no Angular Moderno

Objetivo da aula: Criar um formulário reativo utilizando Angular moderno (Standalone), capturar dados do usuário e integrar com o estado global do carrinho.

Contexto atual da aplicação:

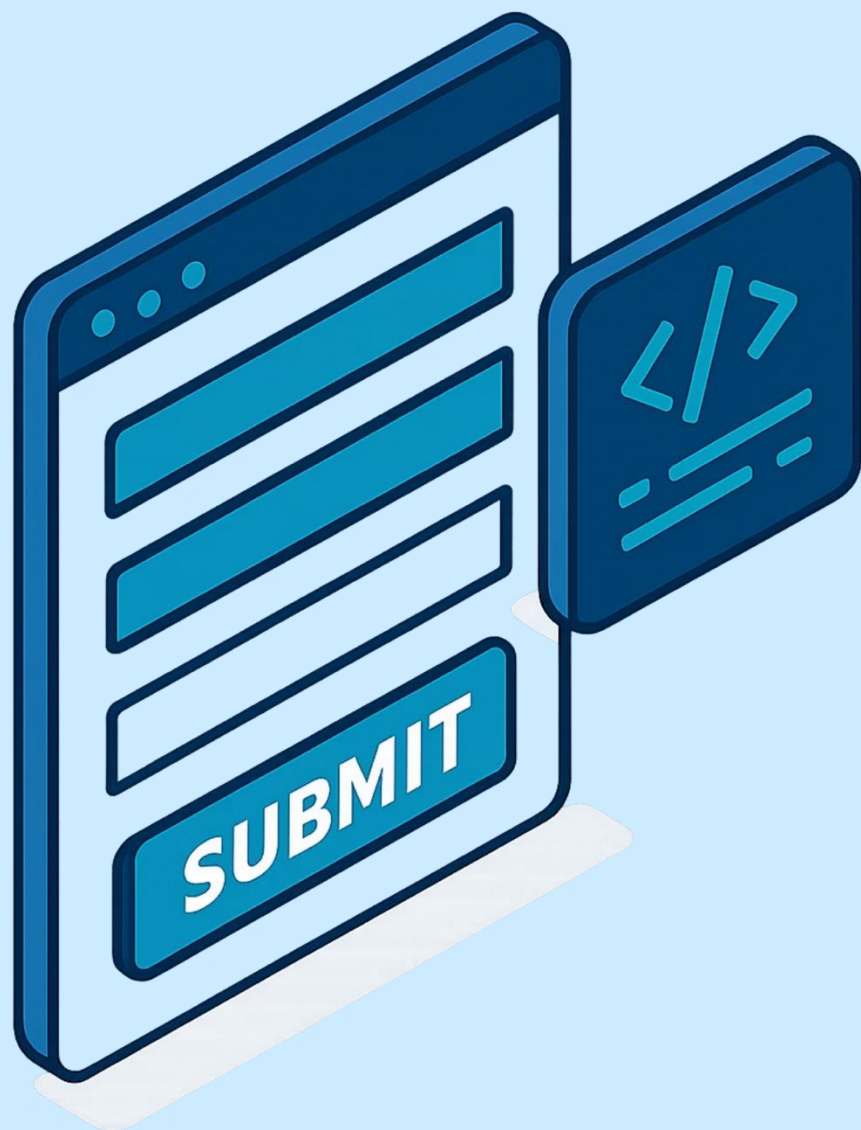
- Listagem de produtos
- Carrinho com estado global (Signals)
- Navegação entre páginas

Problema Atual

A aplicação ainda não possui uma forma de coletar dados do usuário para finalizar uma compra.

Objetivo

Criar uma tela de **checkout** com formulário funcional.



O que são Reactive Forms?

Reactive Forms são uma abordagem do Angular para criação de formulários **baseada em código TypeScript**, oferecendo controle total sobre o comportamento e os dados do formulário.

Características

- Controle total via código
- Estrutura baseada em objetos
- Dados centralizados no componente
- Atualização automática da interface

FormGroup

Representa o formulário completo

FormControl

Representa cada campo individual

Etapa 0 — Parar o Servidor

Ação Obrigatória

CTRL + C

Execute este comando no terminal antes de prosseguir.

Por que é necessário?

- Evita erros durante a criação de arquivos
- Garante que o Angular reconheça corretamente novas estruturas

⚠ Recomendado parar o servidor para evitar inconsistências durante a criação de arquivos.

Etapa 1 – Criar Feature Checkout

Execute o comando abaixo no terminal para gerar o componente de checkout:

```
ng generate component features/checkout/checkout
```

Estrutura criada automaticamente

1

checkout.ts

Lógica do componente em TypeScript

2

checkout.html

Template HTML do formulário

3

checkout.css

Estilos do componente



Caminho completo:

`src/app/features/checkout/checkout/`

O checkout é uma nova funcionalidade da aplicação e deve seguir a **arquitetura baseada em features**, mantendo o código organizado e escalável.

Etapa 2 — Criar Rota do Checkout

Arquivo: `src/app/app.routes.ts`

Adicione a nova rota ao arquivo de rotas da aplicação. Veja o código completo abaixo:

```
import { Routes } from '@angular/router';
import { authGuard } from '../core/auth.guard';

export const routes: Routes = [
  {
    path: '',
    loadComponent: () => import('../features/home/home/home').then((m) => m.Home),
  },
  {
    path: 'produtos',
    loadComponent: () => import('../features/produtos/lista-produtos/lista-produtos').then((m) => m.ListaProdutos),
  },
  {
    path: 'carrinho',
    canActivate: [authGuard],
    loadComponent: () => import('../features/carrinho/carrinho/carrinho').then((m) => m.Carrinho),
  },
  {
    path: 'checkout',
    loadComponent: () => import('../features/checkout/checkout/checkout').then((m) => m.Checkout),
  },
  {
    path: '**',
    redirectTo: '',
  },
];
```

✔ Adicionamos uma nova rota `/checkout` para acessar a tela de checkout da aplicação.

Etapa 3 – Atualizar Menu de Navegação

Arquivo: `src/app/shared/layout/header/header.html`

```
<mat-toolbar color="primary">
  <span>Minha Loja</span>

  <span class="flex-spacer"></span>

  <button mat-button routerLink="/produtos">Produtos</button>
  <button mat-button routerLink="/carrinho">Carrinho ({{ quantidade() }})</button>
  <button mat-button routerLink="/checkout">Checkout</button>
</mat-toolbar>
```

Justificativa: Permitir que o usuário acesse a tela de checkout diretamente através do menu principal da aplicação, tornando a navegação intuitiva e acessível.

Resultado esperado

Um novo item "Checkout" visível no menu de navegação superior da aplicação.

Etapa 4 — Importar ReactiveFormsModule

Arquivo: `src/app/features/checkout/checkout/checkout.ts`

Importações necessárias

```
import { ReactiveFormsModule, FormGroup, FormControl } from '@angular/forms';
```

Adicionar no decorator (em @Component)

```
imports: [ReactiveFormsModule],
```

 **Por que importar explicitamente?** No Angular moderno (Standalone), cada componente precisa importar explicitamente os módulos que utiliza. Não há mais um módulo global que centraliza essas dependências.

ReactiveFormsModule

Habilita diretivas como `formGroup` e `formControlName` no template

Etapa 5 – Criar Formulário e Integração com Carrinho

Arquivo: `src/app/features/checkout/checkout/checkout.ts`

```
import { Component, inject } from '@angular/core';
import { ReactiveFormsModule, FormGroup, FormControl } from '@angular/forms';

import { CarrinhoService } from '../../../core/services/carrinho.service';

@Component({
  selector: 'app-checkout',
  imports: [ReactiveFormsModule],
  templateUrl: './checkout.html',
  styleUrls: ['./checkout.css'],
})
export class Checkout {
  carrinhoService = inject(CarrinhoService);

  formulario = new FormGroup({
    nome: new FormControl(''),
    email: new FormControl(''),
    endereco: new FormControl(''),
  });

  finalizar() {
    console.log('Dados do formulário:', this.formulario.value);
    console.log('Itens do carrinho:', this.carrinhoService.itens());
  }
}
```

FormGroup Representa o formulário completo	FormControl Representa cada campo individual
CarrinhoService Acesso ao estado global do carrinho	finalizar() Captura e exibe os dados no console

Etapa 6 — Criar Template do Formulário

Arquivo: `src/app/features/checkout/checkout/checkout.html`

```
<h2>Checkout</h2>
<form [formGroup]="formulario">
  <p><input type="text" formControlName="nome" placeholder="Nome" /></p>
  <p><input type="email" formControlName="email" placeholder="E-mail" /></p>
  <p><input type="text" formControlName="endereco" placeholder="Endereco" /></p>

  <button type="button" (click)="finalizar()">Finalizar Compra</button>
</form>
```

Conceitos-chave do template

- `formGroup` — conecta o formulário ao TypeScript
- `formControlName` — liga cada campo ao FormGroup
- O botão executa o método `finalizar()`

Etapa 7 — Executar o Projeto

Comando no terminal

```
ng serve
```

Acessar no navegador

```
http://localhost:4200/checkout
```



Validação do Funcionamento

Como testar



Preencher os campos

Insira nome, e-mail e endereço no formulário



Clicar em "Finalizar Compra"

Acione o botão para disparar o método `finalizar()`



Verificar no console

Abra o DevTools (F12) e confira os dados exibidos

Resultado esperado

Dados do formulário exibidos

Nome, e-mail e endereço aparecem no console

Itens do carrinho exibidos

Lista de produtos do carrinho aparece no console

Integração funcionando

Formulário e estado global conectados corretamente





Resultado Final

Após concluir todas as etapas desta aula, a aplicação evoluiu significativamente.

Veja o que foi adicionado:



Tela de Checkout

Nova página dedicada ao processo de finalização de compra



Formulário Reativo

Formulário funcional criado com Reactive Forms do Angular



Captura de Dados

Coleta de nome, e-mail e endereço do usuário



Integração com Estado Global

Formulário conectado ao CarrinhoService via Signals

O que NÃO foi feito nesta aula



⚠ Os tópicos abaixo **não foram abordados** nesta aula e serão tratados na próxima.

Validação de campos

Verificar se os campos foram preenchidos corretamente antes de enviar

Regras de formulário

Definir campos obrigatórios, formatos aceitos e limites de caracteres

Máscaras de entrada

Formatar automaticamente campos como CPF, telefone e CEP



Conclusão da Aula

Parabéns! Nesta aula você dominou os fundamentos dos Reactive Forms no Angular moderno.

O que você aprendeu

- 1 Criar formulários com Reactive Forms
Abordagem baseada em TypeScript para controle total do formulário
- 2 Utilizar FormGroup e FormControl
Estrutura de objetos para representar formulário e campos
- 3 Conectar HTML com TypeScript
Uso de `formGroup` e `formControlName` no template
- 4 Integrar formulário com estado global
Acesso ao `CarrinhoService` dentro do componente de checkout

Dúvidas

?

